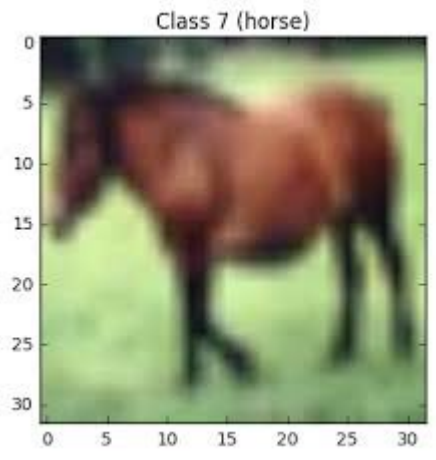
Supervised Convolutional Neural Network

Using CIFAR-10 to Demonstrate Artificial Learning

Alexander Schoolcraft
Richard Zheng
Robert Ward

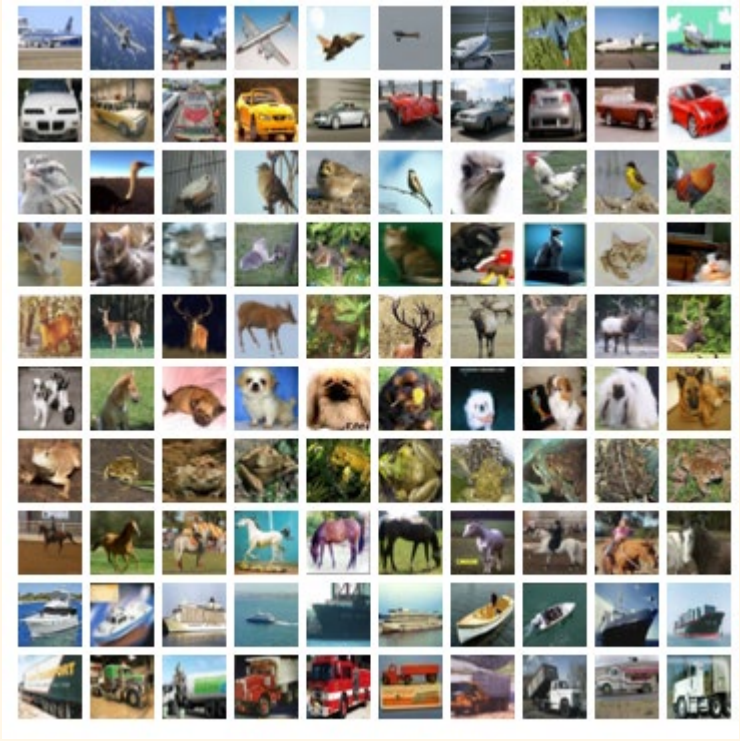
Dataset



CIFAR-10:

60,000 labeled 32*32 full color images in 10 classes

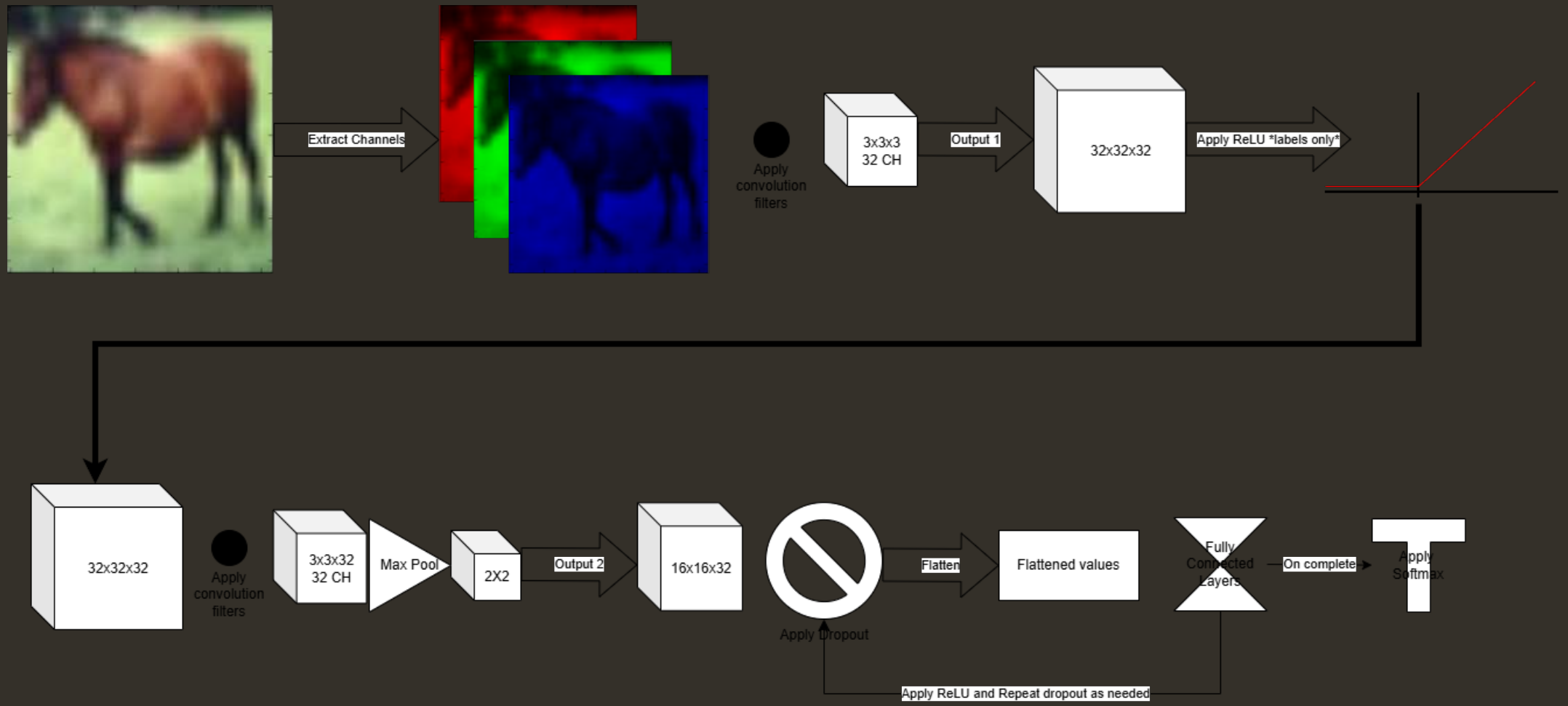
- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck



Model Architecture

This is a rough general overview in these diagrams – we will get to more detail when we get to the code.

Overview



Model Architecture

Feature Extraction

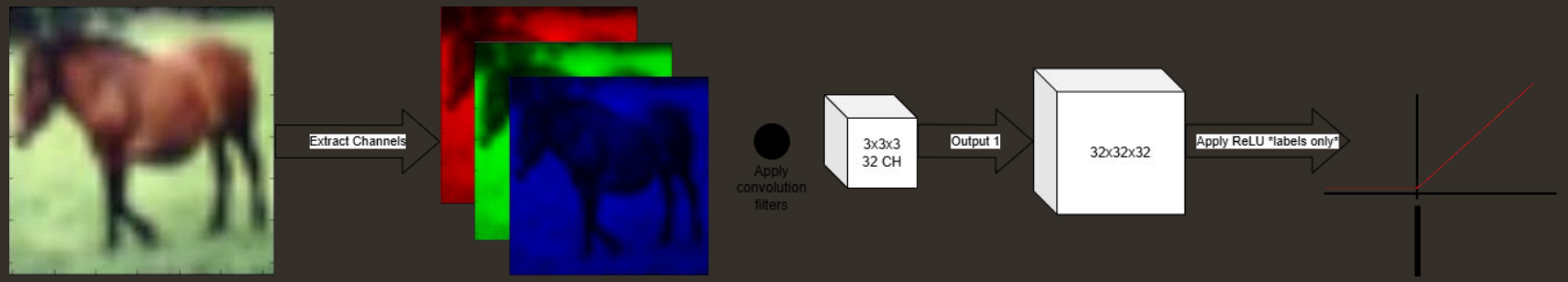
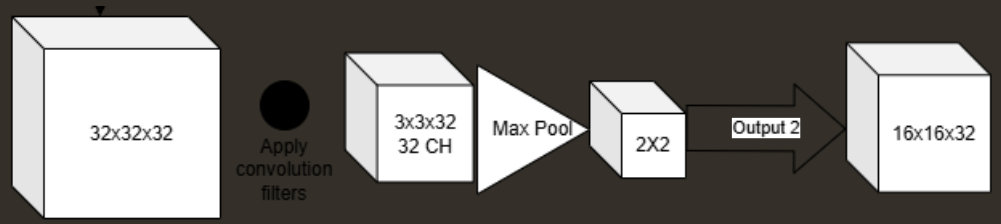


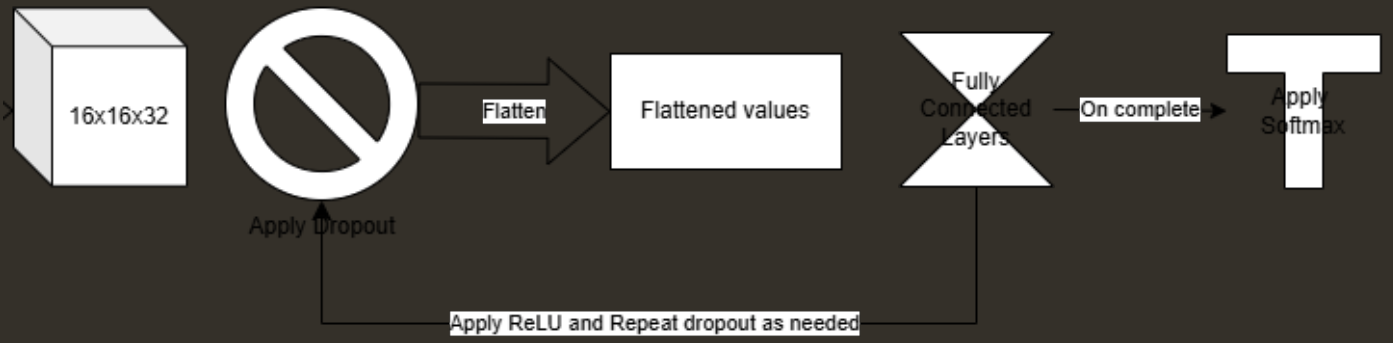
Image is decomposed into its RGB channels and passed through a 32-filter 3×3×3 convolution, producing a 32×32×32 feature block. This block is passed through a ReLU function to introduce non-linearity.



This resultant block is passed through another 32-channel filter, this time 3x3x32, before conducting a 2x2 stride 2 max pooling.

Model Architecture

Neural Network



The resultant feature maps from max pooling then undergo dropout to regularize the network, which the percent dropped is a configurable hyperparameter Can be tuned automatically or specified manually at runtime., default of 30%. This is then flattened into a 4096-unit vector and is fed into a fully connected network.

Our model has 3 fully connected layers, this input layer, then a 256-node layer, then finally, the 10-node output layer. In between the input and ‘hidden’ layer, the model performs ReLU activation and another configurable dropout. For the output layer, the model performs Softmax activation for final classification (which conducts categorical cross-entropy as well)

Model Architecture

The Code

```python
class SmallVGG(nn.Module):
    """A small VGG-style CNN appropriate for 32x32 images like CIFAR-10.
    Three stages of conv blocks separated by MaxPool reduce spatial size.
    """
    def __init__(self, num_classes: int = 10, dropout: float = 0.3):
        super().__init__()
        # Input: [B, 3, 32, 32]
        self.features = nn.Sequential(
            ConvBlock(3,   64, p_drop=dropout/2),
            ConvBlock(64,  64, p_drop=dropout/2),
            nn.MaxPool2d(2),  # -> [B, 64, 16, 16]

            ConvBlock(64,  128, p_drop=dropout/2),
            ConvBlock(128, 128, p_drop=dropout/2),
            nn.MaxPool2d(2),  # -> [B, 128, 8, 8]

            ConvBlock(128, 256, p_drop=dropout),
            ConvBlock(256, 256, p_drop=dropout),
            nn.MaxPool2d(2),  # -> [B, 256, 4, 4]
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),                 # -> [B, 256*4*4]
            nn.Linear(256 * 4 * 4, 512), # hidden layer
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),          # additional classifier regularization
            nn.Linear(512, num_classes), # logits for 10 classes
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.classifier(x)
        return x
```

```python
class ConvBlock(nn.Module):
    """Conv -> BatchNorm -> ReLU -> (optional Dropout) block.
    Using BatchNorm helps stabilize training; Dropout helps regularize.
    """
    def __init__(self, in_ch: int, out_ch: int, p_drop: float = 0.0):
        super().__init__()
        self.conv = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1, bias=False)
        self.bn   = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU(inplace=True)
        self.drop = nn.Dropout(p_drop) if p_drop > 0 else nn.Identity()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.conv(x)
        x = self.bn(x)
        x = self.relu(x)
        x = self.drop(x)
        return x
```

Hyperparameters

Our code had several hyperparameters that could be manually changed on the command line to affect the outcome and to tune for accuracy against overfitting.

- Numerical hyperparameters
  - lr (learning rate) (float)
  - wd (weight decay) (float)
  - dropout (float)
  - batch_size (integer)
  - epochs (integer)
  - patience (integer)

- Categorical Hyperparameters
  - optimizer
    - adamw
    - sgd
  - scheduler
    - cos
    - step
    - onecycle
    - none
  - aug (augmentation)
    - none
    - basic
    - strong

```python
def build_argparser() -> argparse.ArgumentParser:
    """Define CLI to keep the script self-contained and easily configurable."""
    p = argparse.ArgumentParser(description="Train a CIFAR-10 CNN (PyTorch) with optional tuning")
    # Full training options
    p.add_argument("--epochs", type=int, default=30)
    p.add_argument("--batch-size", type=int, default=128)
    p.add_argument("--lr", type=float, default=3e-4)
    p.add_argument("--wd", type=float, default=5e-4)
    p.add_argument("--dropout", type=float, default=0.3)
    p.add_argument("--optimizer", choices=["adamw", "sgd"], default="adamw")
    p.add_argument("--momentum", type=float, default=0.9)
    p.add_argument("--scheduler", choices=["cos", "step", "onecycle", "none"], default="cos")
    p.add_argument("--step-size", type=int, default=30)
    p.add_argument("--gamma", type=float, default=0.1)
    p.add_argument("--aug", choices=["basic", "strong", "none"], default="basic")
    p.add_argument("--patience", type=int, default=10)
    p.add_argument("--label-smoothing", type=float, default=0.0)
    p.add_argument("--workers", type=int, default=4)
    p.add_argument("--data-dir", type=str, default="./data")
    p.add_argument("--out-dir", type=str, default="./outputs")
```

Hyperparameters

In addition to manually tuning hyperparameters, our code had the option to conduct a smaller-scale auto-tune to find the best results.

```python
def build_argparser() -> argparse.ArgumentParser:
    # Tuning controls
    p.add_argument("--auto-tune", action="store_true", help="Run a small random search on a subset before full training")
    p.add_argument("--tune-epochs", type=int, default=4, help="Epochs per tuning trial (keep small)")
    p.add_argument("--tune-max-train", type=int, default=8000, help="Max #training samples used per trial")
    p.add_argument("--tune-batch-size", type=int, default=128, help="Batch size during tuning trials")
    p.add_argument("--tune-trials", type=int, default=12, help="How many random trial configs to try")
```

```python
def tune_hyperparams(base_cfg: TrainConfig) -> TrainConfig:
    """Run a small random search over a tiny subset for a few epochs each.
    Returns a new TrainConfig with the winning hyperparameters applied.
    """
    print("\n===== Hyperparameter Tuning (lightweight) =====")
    set_seed(SEED)

    # We'll reload datasets per-trial to honor different augmentation settings.
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    best_val = -1.0
    best_cfg = None
    tried = 0

    # Iterate random configs
    for trial_cfg in itertools.islice(random_config_space(base_cfg), base_cfg.tune_trials):
        tried += 1
        print(f"\n-- Trial {tried}/{base_cfg.tune_trials} : "
              f"opt={trial_cfg.optimizer} lr={trial_cfg.lr} wd={trial_cfg.weight_decay} "
              f"dropout={trial_cfg.dropout} aug={trial_cfg.aug} sched={trial_cfg.scheduler} "
              f"epochs={trial_cfg.epochs} batch={trial_cfg.batch_size}")

        # Reload datasets to apply the augmentation choice of this trial
        train_set, test_set, val_view = get_datasets(base_cfg.data_dir, aug=trial_cfg.aug)

        # Build small tuning subsets (train and val)
        small_train, small_val = build_tune_subset(train_set, val_view, base_cfg.tune_max_train, seed=SEED)

        # Wrap loaders
        train_loader = DataLoader(small_train, batch_size=trial_cfg.tune_batch_size, shuffle=True,
                                  num_workers=base_cfg.workers, pin_memory=True)
        val_loader   = DataLoader(small_val,   batch_size=trial_cfg.tune_batch_size, shuffle=False,
                                  num_workers=base_cfg.workers, pin_memory=True)

        # Fresh model for each trial
        model = SmallVGG(num_classes=10, dropout=trial_cfg.dropout).to(device)

        # Short training
        metrics = train_validate(model, train_loader, val_loader, trial_cfg, device)
        trial_val = metrics["best_val"]
        print(f"   -> Trial best val_acc = {trial_val*100:.2f}%")

        if trial_val > best_val:
            best_val = trial_val
            best_cfg = trial_cfg

    if best_cfg is None:
        print("No tuning was performed or all trials failed; using base config.")
        return base_cfg
```

```python
    print("\n===== Tuning Complete =====")
    # SAVE the winning configuration to JSON for your report/visibility
    os.makedirs(base_cfg.out_dir, exist_ok=True)
    best_summary = {
        "optimizer": best_cfg.optimizer,
        "lr": best_cfg.lr,
        "weight_decay": best_cfg.weight_decay,
        "dropout": best_cfg.dropout,
        "scheduler": best_cfg.scheduler,
        "augmentation": best_cfg.aug,
        "tuned_val_acc": best_val,
        "tune_epochs": base_cfg.tune_epochs,
        "tune_max_train": base_cfg.tune_max_train,
        "tune_trials": base_cfg.tune_trials,
    }
    with open(os.path.join(base_cfg.out_dir, "tuning_summary.json"), "w", encoding="utf-8") as f:
        json.dump(best_summary, f, indent=2)
    print(f"Saved tuning summary to {os.path.join(base_cfg.out_dir, 'tuning_summary.json')}")

    print(f"Selected: opt={best_cfg.optimizer} lr={best_cfg.lr} wd={best_cfg.weight_decay} "
          f"dropout={best_cfg.dropout} aug={best_cfg.aug} sched={best_cfg.scheduler} "
          f"(val_acc={best_val*100:.2f}%)")

    # Merge the *winning* hyperparameters back into the original training config,
    # but restore full training schedule parameters (epochs, batch_size, patience).
    merged = replace(
        base_cfg,
        lr=best_cfg.lr,
        weight_decay=best_cfg.weight_decay,
        dropout=best_cfg.dropout,
        optimizer=best_cfg.optimizer,
        scheduler=best_cfg.scheduler,
        aug=best_cfg.aug,
        # Keep user's chosen full-training epochs/batch/patience
    )
    return merged
```

Hyperparameters

Finally, there were several hyperparameters that are fixed, and implicit in the design of the neural networks.

- Convolution Filter Size
  - 3x3
    - Small field, “same” padding
- Pooling size and stride
  - 2x2, stride 2
    - Halves width & height
- Activation
  - ReLU
    - Nonlinear activation applied after each convolution and fully connected layer
- Loss function
  - CrossEntropyLoss
    - For multi-classification

Hyperparameters

The results that our run auto-tuned to


```
{
  "optimizer": "adamw",
  "lr": 0.0003,
  "weight_decay": 0.0,
  "dropout": 0.2,
  "scheduler": "onecycle",
  "augmentation": "strong",
  "tuned_val_acc": 0.4055,
  "tune_epochs": 4,
  "tune_max_train": 8000,
  "tune_trials": 20
}
```


- Optimizer: adamw
  - adaptive per-parameter learning rates with built-in weight decay correction
- Scheduler: onecycle
  - rapidly warms up then cools down the LR within one epoch cycle; encourages faster convergence and better generalization
- Augmentation: strong
  - includes flips, small random crops, modest noise, rotations, brightness/contrast jitter, random erasing
- Numericals:
  - Learning Rate: 3e-4
  - Weight Decay: 0.0
  - Dropout: 20%

SLURM Script

```
#!/bin/bash

#SBATCH -p GPU-shared
#SBATCH -t 1:00:00
#SBATCH --gpus=1
#SBATCH --mail-user=schoolcr
#SBATCH --mail-type=ALL

# Safety catch
set -eo pipefail

# Setup
module load anaconda3
source ~/.bashrc || true
conda activate cifar10_cnn
cd /ocean/projects/cis250151p/schoolcr/cifar10

# Tuning run
python cifar10_cnn.py --auto-tune --tune-epochs 4 --tune-trials 20 --tune-max-train 8000 --tune-batch-size 256

# Full run
python cifar10_cnn.py --epochs 100 --batch-size 256 --patience 15
```

Training and Evaluation

Loading the dataset

```python
def get_datasets(data_dir: str, aug: str) -> Tuple[CIFAR10, CIFAR10, CIFAR10]:
    """Download (if needed) and prepare CIFAR-10 training and test datasets."""
    train_tfms, test_tfms = cifar10_transforms(aug=aug)
    train_set = CIFAR10(root=data_dir, train=True, download=True, transform=train_tfms)
    test_set  = CIFAR10(root=data_dir, train=False, download=True, transform=test_tfms)
    # We'll split train_set internally into train/val later; returning train_set/test_set only.
    return train_set, test_set, CIFAR10(root=data_dir, train=True, download=False, transform=test_tfms)  # "val view" uses test_tfms


def split_train_val(train_set: CIFAR10, val_ratio: float = 0.1, seed: int = SEED) -> Tuple[Subset, Subset]:
    """Split the given training dataset into train/val subsets using a fixed seed."""
    num_train = len(train_set)
    num_val = int(num_train * val_ratio)
    # Ensure fixed split across runs for comparability
    generator = torch.Generator().manual_seed(seed)
    train_subset, val_subset = torch.utils.data.random_split(train_set, [num_train - num_val, num_val], generator=generator)
    return train_subset, val_subset


def make_loaders(train_subset, val_subset, batch_size: int, workers: int) -> Tuple[DataLoader, DataLoader]:
    """Wrap subsets in DataLoaders with shuffling for train only."""
    train_loader = DataLoader(train_subset, batch_size=batch_size, shuffle=True,  num_workers=workers, pin_memory=True)
    val_loader   = DataLoader(val_subset,   batch_size=batch_size, shuffle=False, num_workers=workers, pin_memory=True)
    return train_loader, val_loader
```

Training and Evaluation

Manipulating the images

```python
def cifar10_transforms(aug: str = "basic") -> Tuple[transforms.Compose, transforms.Compose]:
    """Return training and testing transforms for CIFAR-10.
    aug: "none" | "basic" | "strong"
    """
    # CIFAR-10 mean/std in RGB order
    mean = (0.4914, 0.4822, 0.4465)
    std  = (0.2023, 0.1994, 0.2010)

    if aug == "none":
        # No augmentation: just normalize
        train_tfms = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize(mean, std),
        ])
    elif aug == "strong":
        # Heavier augmentations to reduce overfitting
        train_tfms = transforms.Compose([
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomCrop(32, padding=4, padding_mode='reflect'),
            transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.02),
            transforms.RandomGrayscale(p=0.05),
            transforms.RandomRotation(10, fill=(int(mean[0]*255), int(mean[1]*255), int(mean[2]*255))),
            transforms.ToTensor(),
            transforms.Normalize(mean, std),
        ])
    else:
        # "basic": widely used CIFAR-10 baseline transforms
        train_tfms = transforms.Compose([
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomCrop(32, padding=4, padding_mode='reflect'),
            transforms.ToTensor(),
            transforms.Normalize(mean, std),
        ])

    # Test/validation transforms never apply stochastic augmentations.
    test_tfms = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(mean, std),
    ])
    return train_tfms, test_tfms
```

Training and Evaluation

Auto-tuning the hyperparameters

```
===== Hyperparameter Tuning (lightweight) =====

-- Trial 1/20 : opt=sgd lr=0.0003 wd=0.0001 dropout=0.4 aug=strong sched=none epochs=4 batch=256
Files already downloaded and verified
Files already downloaded and verified
[001/4] loss=2.3429 train_acc=11.18% val_acc=11.95%
[002/4] loss=2.2507 train_acc=16.14% val_acc=13.85%
[003/4] loss=2.1569 train_acc=20.89% val_acc=15.25%
[004/4] loss=2.0765 train_acc=22.60% val_acc=16.25%
   -> Trial best val_acc = 16.25%

-- Trial 2/20 : opt=sgd lr=0.0003 wd=0.0 dropout=0.3 aug=strong sched=cos epochs=4 batch=256
Files already downloaded and verified
Files already downloaded and verified
[001/4] loss=2.3210 train_acc=11.35% val_acc=14.65%
[002/4] loss=2.2130 train_acc=18.11% val_acc=19.15%
[003/4] loss=2.1453 train_acc=20.67% val_acc=19.20%
[004/4] loss=2.1098 train_acc=21.84% val_acc=18.95%
   -> Trial best val_acc = 19.20%

-- Trial 3/20 : opt=sgd lr=0.0001 wd=0.0005 dropout=0.4 aug=strong sched=onecycle epochs=4 batch=256
Files already downloaded and verified
Files already downloaded and verified
[001/4] loss=2.3856 train_acc=10.40% val_acc=12.05%
[002/4] loss=2.3509 train_acc=10.62% val_acc=15.00%
[003/4] loss=2.3192 train_acc=12.51% val_acc=16.40%
[004/4] loss=2.2995 train_acc=13.54% val_acc=16.35%
   -> Trial best val_acc = 16.40%
```

Training and Evaluation

Final execution

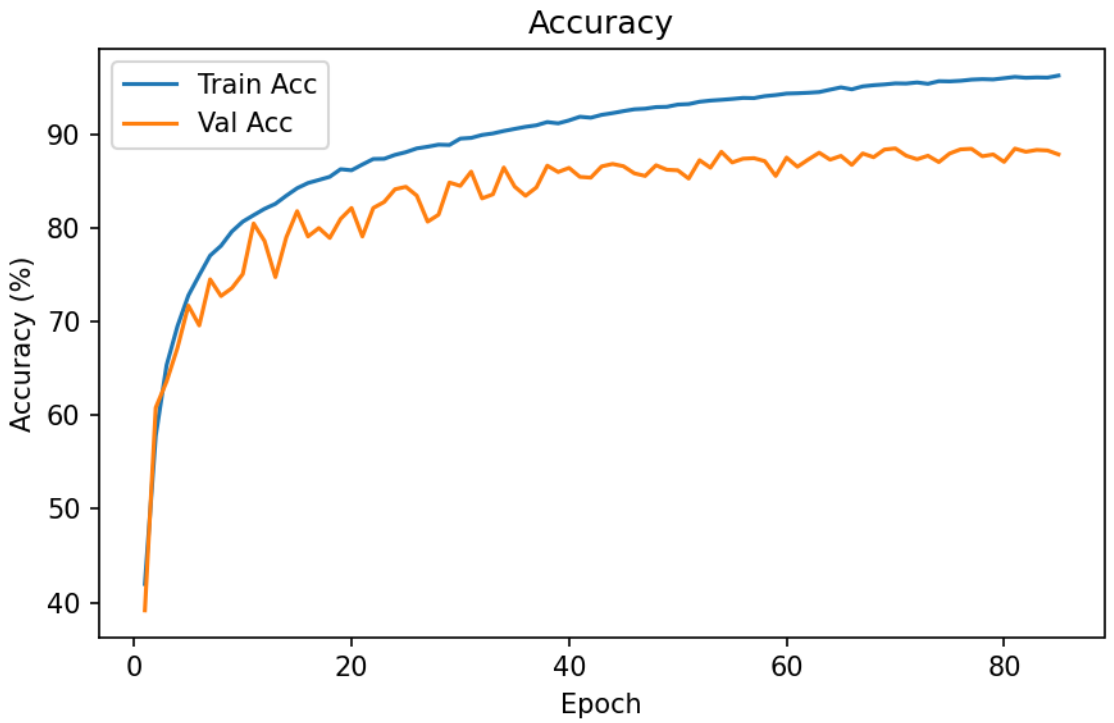
```
[001/100] loss=1.5792 train_acc=41.98% val_acc=39.16%
[002/100] loss=1.1582 train_acc=57.80% val_acc=60.80%
[003/100] loss=0.9661 train_acc=65.34% val_acc=63.60%
[004/100] loss=0.8528 train_acc=69.51% val_acc=67.20%
[005/100] loss=0.7673 train_acc=72.78% val_acc=71.70%
[006/100] loss=0.7070 train_acc=74.95% val_acc=69.58%
[007/100] loss=0.6592 train_acc=77.04% val_acc=74.50%
[008/100] loss=0.6248 train_acc=78.08% val_acc=72.72%
[009/100] loss=0.5862 train_acc=79.61% val_acc=73.56%
[010/100] loss=0.5553 train_acc=80.66% val_acc=75.06%
[011/100] loss=0.5372 train_acc=81.36% val_acc=80.46%
[012/100] loss=0.5161 train_acc=82.03% val_acc=78.60%
[013/100] loss=0.4929 train_acc=82.56% val_acc=74.74%
[014/100] loss=0.4762 train_acc=83.43% val_acc=78.98%
[015/100] loss=0.4541 train_acc=84.23% val_acc=81.78%
[016/100] loss=0.4403 train_acc=84.79% val_acc=79.08%
[017/100] loss=0.4307 train_acc=85.11% val_acc=79.98%
[018/100] loss=0.4182 train_acc=85.45% val_acc=78.92%
[019/100] loss=0.3996 train_acc=86.26% val_acc=80.96%
[020/100] loss=0.3953 train_acc=86.14% val_acc=82.12%
[021/100] loss=0.3808 train_acc=86.76% val_acc=79.08%
[022/100] loss=0.3663 train_acc=87.35% val_acc=82.12%
[023/100] loss=0.3599 train_acc=87.37% val_acc=82.76%
[024/100] loss=0.3501 train_acc=87.79% val_acc=84.12%
[025/100] loss=0.3413 train_acc=88.08% val_acc=84.38%
```

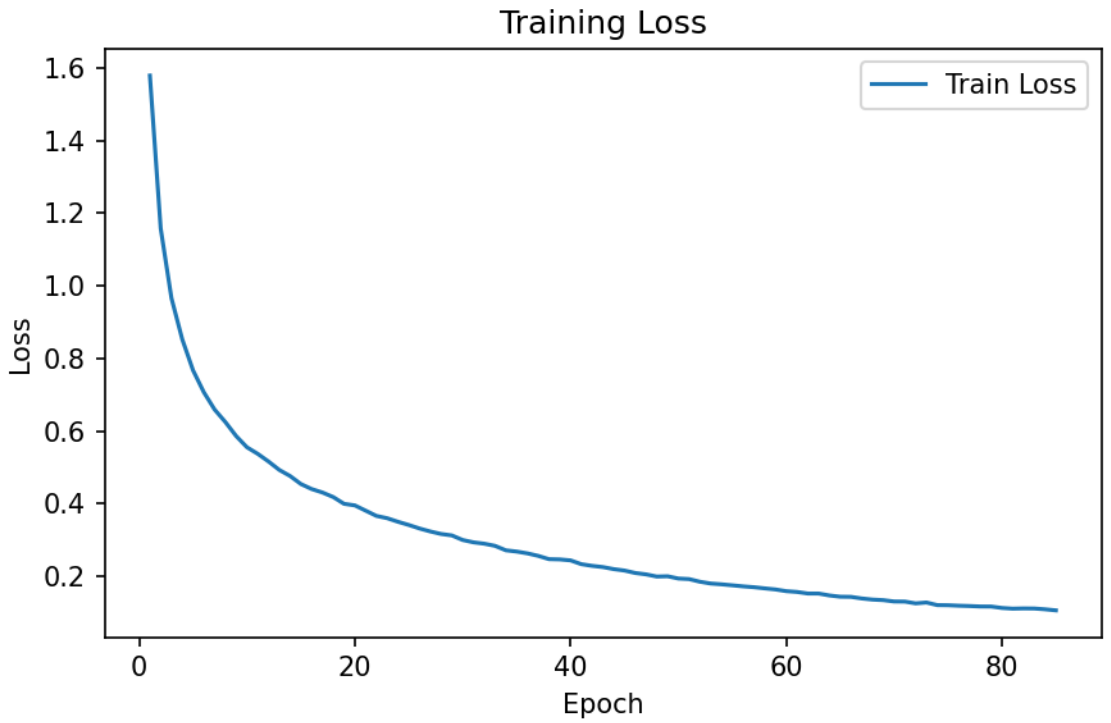
```
[061/100] loss=0.1568 train_acc=94.37% val_acc=86.52%
[062/100] loss=0.1525 train_acc=94.42% val_acc=87.30%
[063/100] loss=0.1524 train_acc=94.50% val_acc=88.02%
[064/100] loss=0.1471 train_acc=94.75% val_acc=87.28%
[065/100] loss=0.1437 train_acc=95.00% val_acc=87.68%
[066/100] loss=0.1433 train_acc=94.79% val_acc=86.72%
[067/100] loss=0.1389 train_acc=95.10% val_acc=87.94%
[068/100] loss=0.1358 train_acc=95.23% val_acc=87.54%
[069/100] loss=0.1342 train_acc=95.31% val_acc=88.36%
[070/100] loss=0.1306 train_acc=95.42% val_acc=88.48%
[071/100] loss=0.1302 train_acc=95.41% val_acc=87.70%
[072/100] loss=0.1253 train_acc=95.52% val_acc=87.32%
[073/100] loss=0.1278 train_acc=95.38% val_acc=87.70%
[074/100] loss=0.1205 train_acc=95.66% val_acc=87.02%
[075/100] loss=0.1202 train_acc=95.64% val_acc=87.94%
[076/100] loss=0.1189 train_acc=95.70% val_acc=88.38%
[077/100] loss=0.1180 train_acc=95.82% val_acc=88.44%
[078/100] loss=0.1168 train_acc=95.87% val_acc=87.64%
[079/100] loss=0.1166 train_acc=95.84% val_acc=87.84%
[080/100] loss=0.1127 train_acc=95.98% val_acc=87.04%
[081/100] loss=0.1108 train_acc=96.11% val_acc=88.46%
[082/100] loss=0.1114 train_acc=96.02% val_acc=88.12%
[083/100] loss=0.1112 train_acc=96.06% val_acc=88.32%
[084/100] loss=0.1090 train_acc=96.04% val_acc=88.26%
[085/100] loss=0.1061 train_acc=96.26% val_acc=87.84%
```

Training and Evaluation

Accuracy and Loss over epochs

```
Early stopping at epoch 85. Best val_acc=88.48%
Loaded best checkpoint with val_acc=88.48%
```



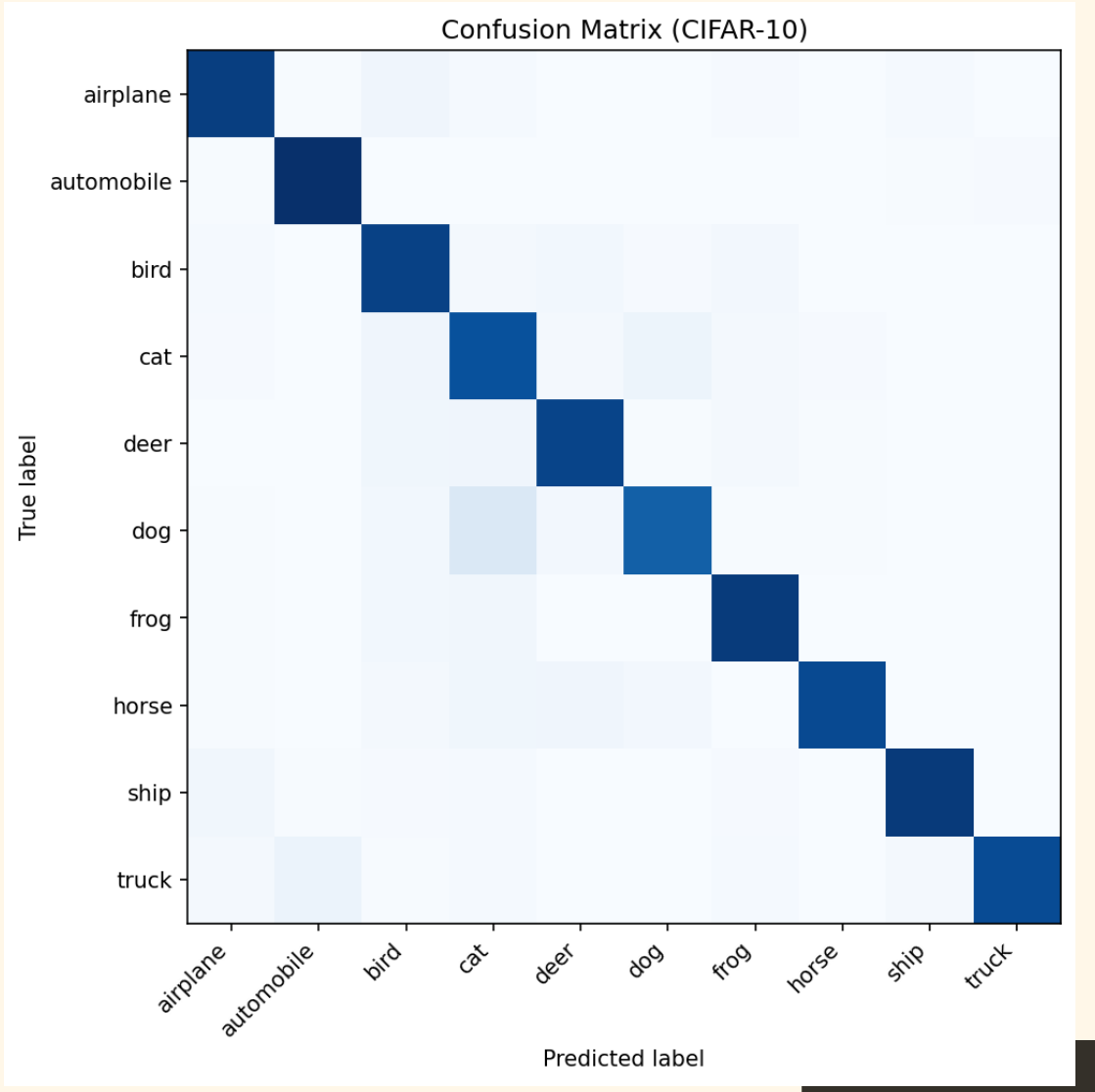


Results

By the Numbers

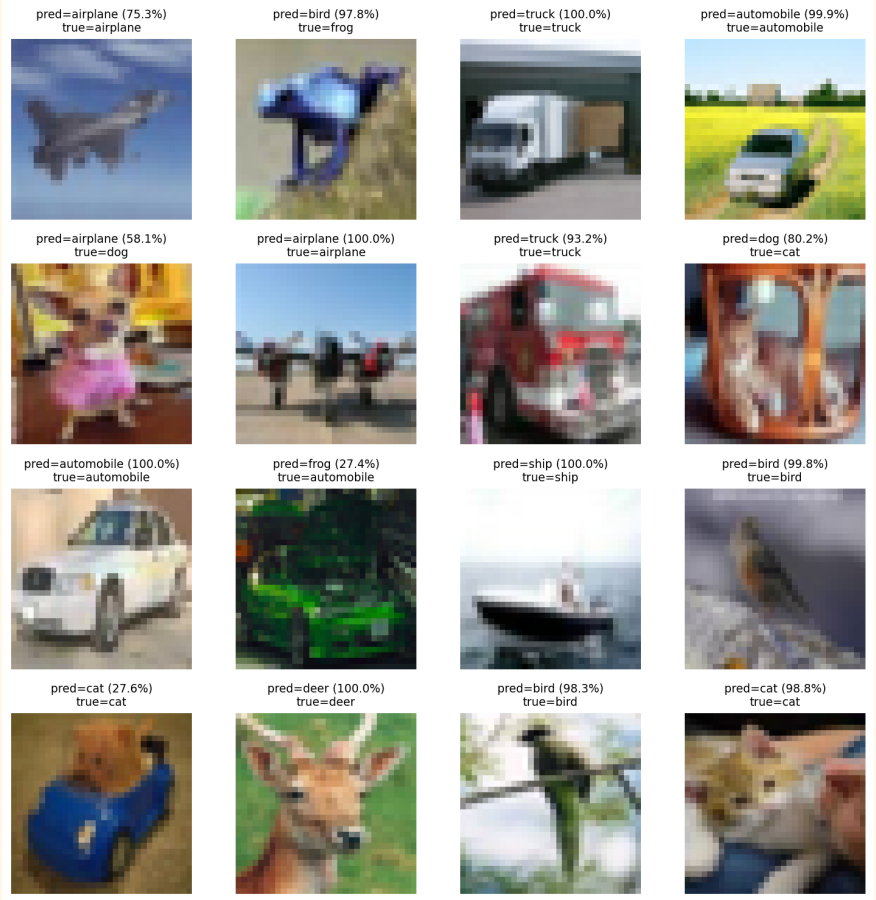
```
Test accuracy: 89.10%
Per-class accuracy:
   0: airplane      91.60%
   1: automobile    97.00%
   2: bird          90.30%
   3: cat           84.70%
   4: deer          89.10%
   5: dog           78.90%
   6: frog          92.60%
   7: horse         87.20%
   8: ship          93.00%
   9: truck         86.60%
```

	precision	recall	f1-score	support
airplane	0.899	0.916	0.907	1000
automobile	0.935	0.970	0.952	1000
bird	0.814	0.903	0.856	1000
cat	0.731	0.847	0.785	1000
deer	0.880	0.891	0.886	1000
dog	0.892	0.789	0.837	1000
frog	0.900	0.926	0.913	1000
horse	0.969	0.872	0.918	1000
ship	0.956	0.930	0.943	1000
truck	0.986	0.866	0.922	1000
accuracy			0.891	10000
macro avg	0.896	0.891	0.892	10000
weighted avg	0.896	0.891	0.892	10000

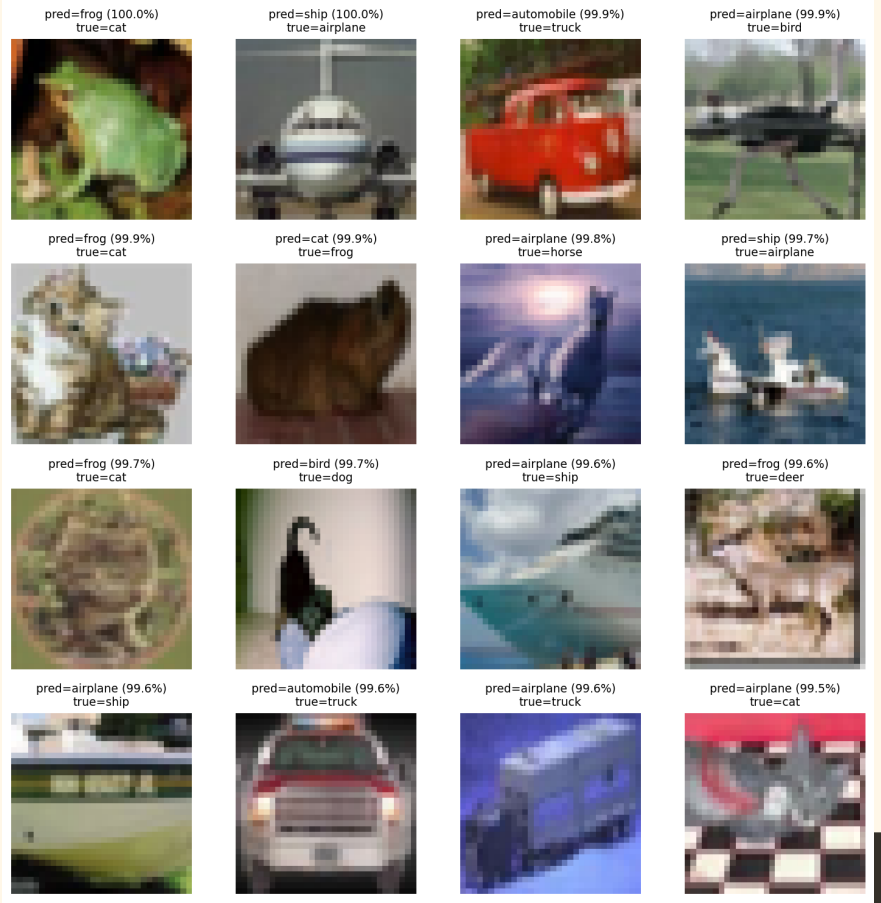


Results

Sample Classifications



Random Samples



Most Incorrect

QUESTIONS?